

```

import nltk
import random

nltk.download('brown')
nltk.download('universal_tagset')

[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.

True

from nltk.corpus import brown

# Extract raw sentences
raw_sentences = brown.sents()

print("Number of sentences:", len(raw_sentences))
print("Sample sentence:", raw_sentences[0])

Number of sentences: 57340
Sample sentence: ['The', 'Fulton', 'County', 'Grand', 'Jury', 'said',
'Friday', 'an', 'investigation', 'of', "Atlanta's", 'recent',
'primary', 'election', 'produced', '``', 'no', 'evidence', "'",
'that', 'any', 'irregularities', 'took', 'place', '.']

# Convert words to lowercase
lower_sentences = [[word.lower() for word in sent] for sent in
raw_sentences]

# Flatten into a single list of tokens
tokens = [word for sent in lower_sentences for word in sent]

print("Total number of tokens:", len(tokens))
print("Sample tokens:", tokens[:20])

Total number of tokens: 1161192
Sample tokens: ['the', 'fulton', 'county', 'grand', 'jury', 'said',
'friday', 'an', 'investigation', 'of', "atlanta's", 'recent',
'primary', 'election', 'produced', '``', 'no', 'evidence', "'",
'that']

# Load tagged sentences with universal tagset
tagged_sentences = brown.tagged_sents(tagset='universal')

print("Sample tagged sentence:")
print(tagged_sentences[0])

Sample tagged sentence:
[('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand',
'ADJ'), ('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an',

```

```
'DET'), ('investigation', 'NOUN'), ('of', 'ADP'), ("Atlanta's",
'NOUN'), ('recent', 'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'),
('produced', 'VERB'), ('`', '.'), ('no', 'DET'), ('evidence',
'NOUN'), ('"', '.'), ('that', 'ADP'), ('any', 'DET'),
('irregularities', 'NOUN'), ('took', 'VERB'), ('place', 'NOUN'), ('.',
'.')] ]
```

```
# Lowercase words in tagged sentences
```

```
tagged_sentences_lower = [
    [(word.lower(), tag) for word, tag in sent]
    for sent in tagged_sentences
]
```

```
random.shuffle(tagged_sentences_lower)
```

```
split_point = int(0.9 * len(tagged_sentences_lower))
train_data = tagged_sentences_lower[:split_point]
test_data = tagged_sentences_lower[split_point:]
```

```
from nltk.tag import HiddenMarkovModelTrainer
```

```
trainer = HiddenMarkovModelTrainer()
```

```
# Train supervised HMM
```

```
hmm_tagger = trainer.train_supervised(train_data)
```

```
print("Initial POS probabilities:")
```

```
print(hmm_tagger._priors)
```

```
Initial POS probabilities:
<MLEProbDist based on 51606 samples>
```

```
print("\nPOS Transition Probabilities:")
```

```
print(hmm_tagger._transitions)
```

```
POS Transition Probabilities:
<ConditionalProbDist with 12 conditions>
```

```
print("\nEmission Probabilities:")
```

```
print(hmm_tagger._outputs)
```

```
Emission Probabilities:
<ConditionalProbDist with 12 conditions>
```

```
def predict_pos(sentence):
```

```
    """
```

```
        Predict POS tags for a sentence
```

```
    """
```

```

    sentence = [word.lower() for word in sentence]
    return list(zip(sentence, hmm_tagger.tag(sentence)))

test_sentence = "The quick brown fox jumps over the lazy dog".split()

prediction = predict_pos(test_sentence)

for word, tag in prediction:
    print(f"{word:10s} -> {tag}")

/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:335:
RuntimeWarning: overflow encountered in cast
    0[i, k] = self._output_logprob(si, self._symbols[k])

the      -> ('the', 'DET')
quick    -> ('quick', 'ADJ')
brown    -> ('brown', 'NOUN')
fox       -> ('fox', 'NOUN')
jumps    -> ('jumps', 'VERB')
over     -> ('over', 'ADP')
the      -> ('the', 'DET')
lazy     -> ('lazy', 'ADJ')
dog      -> ('dog', 'NOUN')

accuracy = hmm_tagger.evaluate(test_data)
print("Model Accuracy:", accuracy)

/tmp/ipython-input-2122169639.py:1: DeprecationWarning:
  Function evaluate() has been deprecated. Use accuracy(gold)
  instead.
    accuracy = hmm_tagger.evaluate(test_data)
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:363:
RuntimeWarning: overflow encountered in cast
    0[i, k] = self._output_logprob(si, self._symbols[k])

Model Accuracy: 0.7938587442404236

```